ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) | 979-8-3503-6874-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICASSP49660.2025.10889184

Multi-source Data Lossless Compression via Parallel Expansion Mapping and xLSTM

Huidong Ma[1], Hui Sun[1, 2], Liping Yi[1, 2], Xiaoguang Liu[1, *], Gang Wang[1, *]
[1]College of Computer Science, NBJL, ICIC, TMCC, SysNet, DISSec, GTIISC, Nankai University, Tianjin 300350, China
[2]College of Computing and Data Science (CCDS), Nanyang Technological University, Nanyang 639798, Singapore

***Abstract*—Explosive growth of multi-source data (MSD) poses challenges in data transmitting and storing. Neural Network (NN)-based lossless compressors are an important type of compression approaches to alleviate these problems. However, existing NN-based lossless compressors suffer from poor compression ratio and high time cost at the same time. To address these issues, we propose a novel MSD Lossless Compressor (MSDLC) with two compression stages: 1) We propose a Parallel Expansion Mapper (PEM) to map redundant pieces in MSD into unused alphabet values, which not only compresses MSD but also saves time for the next stage's NN-based lossless compression. 2) With the mapped MSD as input, we design a NN-based lossless compressor to further improve compression ratio, where we introduce the state-of-the-art xLSTM model and design a Deep Spatial Gating Module (DSGM) as the backbone of NN. We compare MSDLC with 11 baselines on 6 real-world datasets and the results validate that MSDLC obtains the best average compression ratio and time cost. Compared with baselines, compression ratios are improved by 1.103%∼113.897%, and the time costs are improved by 41.367%∼73.891%. The codes can be available at https://github.com/mhuidong/MSDLC.**

***Index Terms*—Multi-source Data, Lossless Compression, Parallel Expansion Mapping, xLSTM, Deep Spatial Gating Module**

I. Introduction

Multi-Source data (MSD) [1] consist of image, video, text, etc. The massive amount of MSD poses significant challenges in data transmitting and storing [2], [3]. Compressing MSD is an natural solution. Traditional dictionary-based compressors [4]–[8] have been validated with poor compression ratios on MSD due to the insufficient utilization of MSD redundancy.

Recently, NN-based lossless compressors like DZip [9], TRACE [10], and PAC [11] show great potential in MSD compression owing to powerful capabilities in context modeling. However, these methods still face two challenges:

- **Poor Compression Ratio.** They use common deep learning models (such as recurrent neural network [12]) as predictors to predict the probability distribution, but these models can not learn redundant information in the data very well, resulting in inferior compression ratios.
- **High Time Cost.** They consume high time cost by two aspects: a) introducing NN increases the computational overhead of compression and decompression, b) not supporting multi-GPU parallelism.

* Corresponding authors: {liuxg, wgzwp}@nbjl.nankai.edu.cn.

This research is supported in part by the National Science Foundation of China under Grant 62272252 & 62272253, and the Fundamental Research Funds for the Central Universities.

To solve these problems, we propose a novel **MSD L**ossless **C**ompressor (**MSDLC**) with the following three contributions:

- We propose a novel Parallel Expansion Mapper (PEM) to map frequently occurring pieces in MSD into unused alphabet values, which reduces sequence length and then saves time for the next NN-based compression stage.
- We propose a novel NN compression predictor, consisting of the state-of-the-art xLSTM and a well-designed Deep Spatial Gating Module (DSGM) learning cross-space symbol interaction information, which effectively improves compression ratio.
- We compare MSDLC with 11 baselines on 6 real-world datasets, experiment results show that MSDLC outperforms baselines in both compression ratio and time cost, with a 1.1%∼113.9% improved compression ratios and a 41.37%∼73.89% saved running time costs.

II. Related work

Existing NN-based MSD compressors can be divided into static or dynamic according to whether the prediction model of the compressor is trained or not during compression process.

Static. Their compression models are first pre-trained on the source file and then are frozen and used to compress data directly. DeepZip [13] and DecMac [14] use RNN [12] and LSTM [15] as predictors, respectively, but they suffer from long pre-training time. With the development of large language models (LLM) [16], LLM-based lossless compressors have recently emerged. LLMZip [17] and LMIC [18] use Chinchilla [19] and LLaMa [20] as predictors, respectively, but they are more time-consuming due to larger scales of LLM parameters even if achieving better compression ratios.

Dynamic. They train compression models while compressing data, enabling the model to capture contextual relationships in MSD sequences. CMix [21] mixes the predictions from over two thousand of models to obtain a strong compression effect, but it has significantly low compression speed. NNCP [22], lstm-compress [23] and DZip [9] use LSTM and corresponding variants as predictors. TRACE [10] uses Performer [24], a variant of Transformer [25], as the predictor. OREO [26] and PAC [11] utilize a multilayer perceptron (MLP) model to achieve compression while reducing computational costs.

All above methods face inferior balance of compression effects and time. Thus, we propose MSDLC based on proposed PEM and xLSTM architecture with a designed DSGM, effectively balancing compression ratio and time.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 07:45:56 UTC from IEEE Xplore. Restrictions apply.

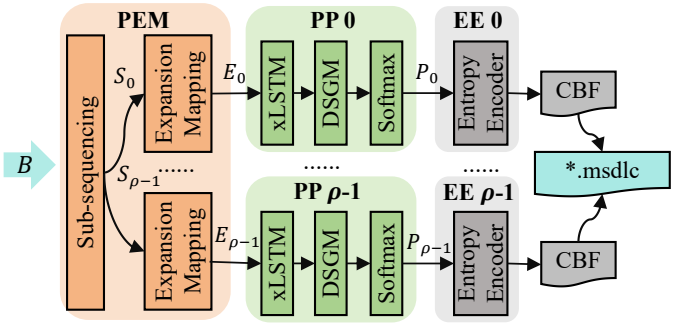


Fig. 1. Compression pipeline of the MSDLC. S, E denote the inputted and mapped sub-sequence, respectively. P represents the probability distribution. CBF indicate the compressed binary file.

III. Method

Fig. 1 shows the workflow of MSDLC compression, including three important components and corresponding operations:

- **Parallel Expansion Mapper (PEM).** To reduce compression time, PEM first divides the inputted MSD byte streams B into ρ sub-sequences $\{S_i\}_{i=0}^{\rho-1}$ with the same length. Then it further splits each S_i into multiple pieces, and maps high-frequency pieces into unused alphabet values. Last it forms a shorter mapped sequence E_i with all mapped and unmapped pieces, where $i = 0, ..., \rho - 1$.
- **Probability Predictor (PP).** To improve compression ratio, for each mapped sequence E_i, PP inputs it into a xLSTM model followed by a designed Deep Spatial Gating Module (DSGM) and a Softmax layer to output probability distributions P_i, where $i = 0, ..., \rho - 1$.
- **Entropy Encoder (EE).** EE uses an arithmetic encoder [27] to compress each mapped sequence E_i into a compressed binary stream file based on P_i, and it packages all the binary files into a final compressed file with an extension $*.msdlc$, where $i = 0, ..., \rho - 1$.

Since PEM inherently enables data-parallelism, MSDLC can support SIMD (Single In Multiple Data) [28]–[30] parallelism on multiple GPUs for acceleration.

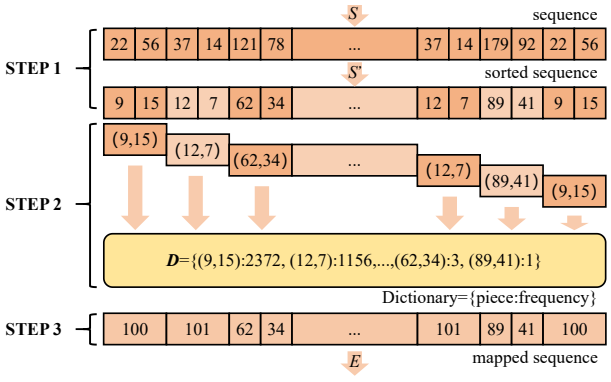


Fig. 2. Workflow of the proposed Parallel Expansion Mapper (PEM). S, S' and E denote the inputted, sorted, and mapped MSD sequence, respectively. D is a dictionary for storing MSD pieces and corresponding frequencies.

A. Parallel Expansion Mapper (PEM)

In MSD compression, the original MSD is read as a byte stream with a alphabet size $|A| = 256$ to simplify computation. However, some MSD has a alphabet size smaller than 256, such as DNA [31] with only $\{A, C, G, T\}$ symbols. In a DNA sequence, the adjacent symbols can be seen as pieces, and we can encode them into shorter unused alphabet values to reduce the length of MSD. As shown in Fig. 2, there are 3 steps to encode each sub-sequence S to the mapped sequence E.

Step1: Counting the number v of unique single symbols in the input sub-sequence S, and then sort them into the first v values of the alphabet A with a length of 256.

Step2: Using a sliding window with a size of $\max(\lfloor \log_v(|A|-v) \rfloor, 2)$ and a stride with the same length as the window size to split multiple pieces.

Step3: Analyzing the frequency of unique pieces, and select top-$(|A| - v)$ high-frequency pieces to map them into the remaining $|A| - v$ alphabet values.

B. Probability Predictor (PP)

PP uses a historical symbol collection with fixed-length to predict the probability of the next occurred symbol (also called target symbol), and the predicted probability of the target symbol is then compressed by the entropy encoder into a binary file. Therefore, the inference accuracy and the computation latency of the predictor directly affect compression ratio and running time. The structure of PP is shown in Fig. 3, it consists of three crucial parts.

xLSTM. It is a recently proposed state-of-the-art model [32] for processing and modeling sequence data. Based on the traditional LSTM, it introduces two innovative modules: exponential gating mechanisms (sLSTM) and matrix memory structures (mLSTM). Compared with the traditional LSTM, it enhances the memory function of LSTM and also supports parallel computations that are not incompatible with LSTM. We use it as a main component of the probability predictor.

Deep Spatial Gating Module (DSGM). Although increasing xLSTM layers improves prediction accuracy, the incurred computational cost is high. To address it, we design a Deep Spatial Gating Module (DSGM) for better capturing interaction information cross symbols at a low computational cost.

The mapped sub-sequence E from PEM is inputted into the xLSTM to output $X \in \mathbb{R}^{t \times d}$, ($t$: timestep, d: embedding dimension), which is then passed through the designed DSGM.

X is first mapped to a higher-dimensional representation as

$$H_\alpha = \sigma(XW_\alpha + b_\alpha), W_\alpha \in \mathbb{R}^{d \times 2h}, b_\alpha \in \mathbb{R}^{1 \times 2h}, \tag{1}$$

where h is the dimension of the hidden layer, σ is an activation function GELU [33], $H_\alpha \in \mathbb{R}^{t \times 2h}$.

H_α is then inputted into a spatial gating unit [34], which decomposes H_α into two independent parts U and V by taking the innermost layer as the split point. Then we can obtain $H_\beta \in \mathbb{R}^{t \times h}$ by:

$$\begin{aligned} U &= H_\alpha[:, 0:h], U \in \mathbb{R}^{t \times h}, \\ V &= H_\alpha[:, h:2h], V \in \mathbb{R}^{t \times h}, \end{aligned} \tag{2}$$

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 07:45:56 UTC from IEEE Xplore. Restrictions apply.

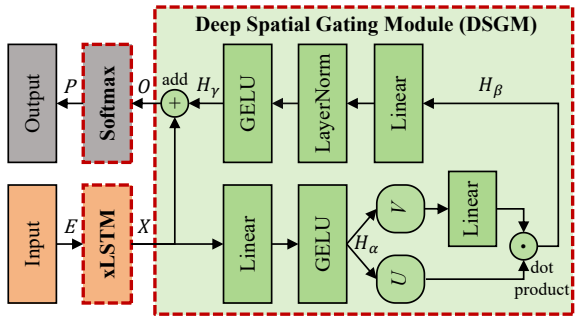


Fig. 3. Workflow of the designed Probability Predictor (PP). E represents the mapped sequence. X and O denote the output of xLSTM and logits of DSGM, respectively. P indicates the probability distribution. H_α, H_β and H_γ represent the outputs of each hidden layer.

$$H_\beta = U \odot (W_\beta V + b_\beta), W_\beta \in \mathbb{R}^{t\times t}, b_\beta \in \mathbb{R}^{1\times h}, \tag{3}$$

where $\odot$ is the dot product operation.

Next, H_β is mapped to $H_\gamma \in \mathbb{R}^{t\times d}$ by

$$H_\gamma = \sigma(\delta(H_\beta W_\gamma + b_\gamma)), W_\gamma \in \mathbb{R}^{h\times d}, b_\gamma \in \mathbb{R}^{1\times d}, \tag{4}$$

where δ represents the LayerNorm layer.

H_γ is combined with X through a residual connection to obtain the output logits O as

$$O = \frac{H_\gamma + X}{2}. \tag{5}$$

In short, compared with simply stacking xLSTM layers, DSGM better learns contextual relationships cross symbols by higher representation processing and spatial gating units with stronger memory, which not only improves model prediction performances but also consumes low computational cost due to only linear operations that can be accelerated in parallel.

Softmax. The output logits O from the DSGM is processed by a Softmax layer to output corresponding probability P of the target symbol, which is then compressed into a binary file through the entropy encoder.

IV. Experiments.

All experiments are conducted on a Ubuntu server with 4 Intel Xeon Silver 4310 CPUs, 4 NVIDIA GeForce RTX 4090 GPUs, and 128 GB of DDR4 RAM.

A. Setup

Datasets. We evaluate compression algorithms on 6 real-world datasets following [9]–[11]: 2 text-numerical datasets (SAO and X-RAY) [35], 2 image datasets (MR and NCI) [35], 2 genomics datasets (HAHI and DRME) [36]. The total data size of all datasets is 95,321,627 bytes.

Metrics. Following [37]–[42], we evaluate compressors by two metrics: compression ratio (*CR*) and running time (*RT*). Among them, the *CR* is defined as:

$$CR = \frac{Size_{compressed}}{Size_{source}} \times 8\ bits/base, \tag{6}$$

where the file size is recorded in bytes. Smaller values of *CR* and *RT* mean better compression performances.

Baselines. We compare MSDLC with 7 traditional compressors (Gzip [43], PBzip2 [44], XZ [45], SnZip [46], Lzma2 [47], PPMD [48], and LZ4 [49]) and 4 state-of-the-art open-sourced NN-based lossless compressors (DeepZip [13], DZip [9], TRACE [10] and PAC [11]).

Parameters Setting. Following hyperparameter tuning in TRACE [10], we adjust four key hyperparameters to enable MSDLC to make the best balance between compression ratio and running time: the number of network layers is 3, embedding dimension is 256, batch size is 512, and timestep is 32.

B. Compression Ratio

Table I shows the *CR* (bits/base) of MSDLC and baselines on 6 datasets. MSDLC outperforms all traditional and NN-based baselines on all datasets, except the DRME dataset where the *CR* is second only to the PAC. Compared with baselines, MSDLC increases the average *CR* improvement ($\frac{CR(baseline)-CR(ours)}{CR(baseline)} \times 100\%$) by up to 113.897% (SnZip). These significant improvements are attributed to the predictor with xLSTM and DSGM which provide better learning ability in redundant MSD features.

The slightly lower *CR* of MSDLC than PAC on the DRME may be because this dataset is relatively simple, the MLP model in PAC is sufficient to learn the MSD distribution, while the PP in MSDLC may not have been learned sufficiently.

We also see that the static NN-based compressors DeepZip and DZip, using a pre-trained model to compress data, perform a worse average *CR* than MSDLC (101.883% improvement on DeepZip, and 101.539% improvement on DZip). Besides, we also find that the size of compressed files in DeepZip and DZip is even larger than the source data file because the pre-trained static models also need to be stored as a part of the compressed file.

C. Running Time

Table II shows the *RT* (Minutes) of MSDLC and 4 NN-based lossless compressors: DeepZip, DZip, TRACE and PAC on 6 datasets. The results show that MSDLC consumes the lowest computational time, with a 41.369%-73.891% average time saved. This reflects that the piece mapping and the inherent parallel mechanism achieved by the expansion mapper effectively improve compression efficiency. However, other NN-based lossless compressors leave many alphabet values unused, leading to the high redundancy of the data input into the predictor, and the lack of parallel compression also results in high compression time.

D. Ablation Study

Table III shows the average *CR* and *RT* of ablation study on 6 datasets. We find that introducing DSGM improves *CR*, while further introducing PEM slightly worsens *CR*, because the dynamic prediction model faces a cold-start problem which is exacerbated by parallel compression. However, introducing PEM significantly reduces time costs. These results validate the necessity of all components in MSDLC.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 07:45:56 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPRESSION RATIO (BITS/BASE) OF MSDLC AND BASELINES. THE BOLDFACE MEANS THE BEST RESULT.

Method	SAO	X-RAY	MR	NCI	HAHI	DRME	Average	*CR*-Improvment
Gzip [43]	5.877	5.700	2.948	0.712	2.203	2.220	3.488	51.702 (%)
PBzip2 [44]	5.452	3.824	1.971	0.441	2.120	2.159	2.762	20.109 (%)
XZ [45]	4.882	4.239	2.208	0.345	2.029	1.993	2.741	19.196 (%)
SnZip [46]	7.102	7.985	4.333	1.460	3.710	3.694	4.918	113.897 (%)
Lzma2 [47]	4.869	4.229	2.205	0.415	2.025	1.993	2.749	19.544 (%)
PPMD [48]	5.277	3.635	1.862	0.353	2.035	2.054	2.632	14.490 (%)
LZ4 [49]	6.256	6.778	3.364	0.865	2.766	2.822	4.006	74.223 (%)
DeepZip [13]	8.506	7.135	4.647	0.854	2.067	1.916	4.642	101.883 (%)
DZip [9]	8.503	7.118	4.635	0.849	2.064	1.913	4.634	101.539 (%)
TRACE [10]	4.271	3.432	1.682	0.361	1.877	1.913	2.325	1.103 (%)
PAC [11]	4.264	3.456	1.791	0.347	1.865	**1.898**	2.345	1.973 (%)
MSDLC (Ours)	**4.262**	**3.414**	**1.623**	**0.333**	**1.864**	1.912	**2.299**	-

TABLE II
RUNNING TIME (MINUTES) OF MSDLC AND BASELINES. THE BOLDFACE MEANS THE BEST RESULT.

Method	SAO	X-RAY	MR	NCI	HAHI	DRME	Average	*RT*-Saving
DeepZip [13]	14.300	16.583	19.083	60.167	3.635	28.001	23.628	44.558 (%)
DZip [9]	30.383	35.350	39.567	124.967	7.314	63.466	50.174	73.891 (%)
TRACE [10]	12.133	14.000	15.400	46.400	5.634	40.486	22.342	41.367 (%)
PAC [11]	13.883	15.983	17.583	52.750	5.452	39.303	24.159	45.776 (%)
MSDLC (Ours)	**11.169**	**13.052**	**14.702**	**24.918**	**1.884**	**13.068**	**13.132**	-

TABLE III
ABLATION AVERAGE RESULTS ON 6 DATASETS.

Combination	xLSTM	DSGM	PEM	*CR*	*RT*
MSDLC-A	✓	✗	✗	2.286	43.032
MSDLC-B	✓	✓	✗	**2.272**	45.654
MSDLC-C	✓	✓	✓	2.299	**13.132**

E. Impacts of Layers and Embedding Dimensions

CR Improvement Cost (*CIC*) is a metric that quantifies the balance of compression ratio and time cost, defined as [10]:

$$CIC_{i,j} = (CR_{i,j} \times CR_{base}) \frac{T_{i,j} - T_{base}}{CR_{base} - CR_{i,j}}, \tag{7}$$

where $CR_{i,j}$ and $T_{i,j}$ represent the compression ratio and running time of compressors using i layers and a j-dimensional embedding size, CR_{base} and T_{base} are the situation with the lowest compression ratio, we set $base = 2, 32$. A higher *CIC* means a higher time cost for compression.

We change the number of layers and embedding dimensions to calculate and normalize *CIC* values on SAO dataset. Fig. 4(a) shows that increasing the number of layers incurs more time costs than increasing the embedding size. To obtain a better *CR* with an acceptable time overhead, we set the number of layers to 3 and the embedding dimension to 256.

F. Impact of Timestep

We conduct experiments to test the impact of timestep in the compression ratio on SAO and X-RAY datasets, and we set the same batch size 512 as TRACE [10]. Fig. 4(b) shows that MSDLC has the optimal *CR* when timestep is set to 32.

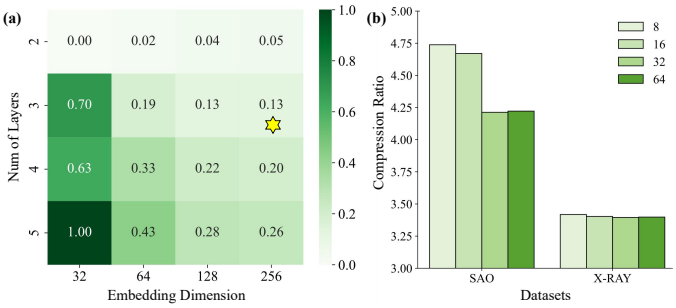


Fig. 4. Impacts of the number of layers, embedding dimension, and timestep.

V. CONCLUSION

This paper proposes a multi-source data (MSD) lossless compressor MSDLC. The designed Parallel Expansion Mapping (PEM) divide MSD into multiple sub-sequences for parallel processing to save running time, and it also maps MSD data to shorter alphabet values for reducing redundancy to improve compression ratio. The designed Probability Predictor (PP) introduces the state-of-the-art model xLSTM and a well-designed Deep Spatial Gating Module (DSGM) for enhancing model prediction capability to improve compression ratios. Experiments on 6 real-world datasets demonstrate that MSDLC outperforms 11 advanced compressors in both compression ratio and running time.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 07:45:56 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Sun Hui and Ma Huidong. Neural-network-based lossless universal compressions benchmark, 2024. [Online]. Available: https://fahaihi.github.io/NNLCB.

[2] John F.Gantz. David Reinsel John Rydning. The world keeps creating more data — now, what do we do with it all? https://www.idc.com/getdoc.jsp?containerId=US46410421, 2021.

[3] John Rydning. Data marketplaces, and data as a service. https://www.idc.com/getdoc.jsp?containerId=IDC_P38353, 2023.

[4] Ipavlov. A free file archiver for extremely high compression, 2024. [Online]. Available: https://www.7- zip.org.

[5] Brotli. Brotli official website, 2024. [Online]. Available: https://github.com/google/brotli.

[6] Zstd. Zstd official website, 2024. [Online]. Available: https://facebook.github.io/zstd/.

[7] Ilya Grebnov. Bsc official website. https://github.com/IlyaGrebnov/libbsc.

[8] Julian Seward. The official website of the xz compressor, 2024.

[9] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. Dzip: Improved general-purpose loss less compression based on novel neural network modeling. In *2021 Data Compression Conference (DCC)*, pages 153–162. IEEE, 2021.

[10] Yu Mao, Yufei Cui, Tei-Wei Kuo, and Chun Jason Xue. Trace: A fast transformer-based general-purpose lossless compressor. In *Proceedings of the ACM Web Conference 2022*, pages 1829–1838, 2022.

[11] Yu Mao, Jingzong Li, Yufei Cui, and Jason Chun Xue. Faster and stronger lossless compression with optimized autoregressive framework. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.

[12] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404:132306, 2020.

[13] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. Deepzip: Lossless data compression using recurrent neural networks. *arXiv preprint arXiv:1811.08162*, 2018.

[14] Q Liu, Y Xu, and Z DecMac Li. A deep context model for high efficiency arithmetic coding. In *Proceedings of the International Conference on Artificial Intelligence in Information and Communication (ICAIIC), Okinawa, Japan*, pages 11–13, 2019.

[15] Hochreiter Sepp and Schmidhuber Jürgen. Long short-term memory. *Neural Computation MIT-Press*, 1997.

[16] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):1–26, 2024.

[17] Chandra Shekhara Kaushik Valmeekam, Krishna Narayanan, Dileep Kalathil, Jean-Francois Chamberland, and Srinivas Shakkottai. Llmzip: Lossless text compression using large language models. *arXiv preprint arXiv:2306.04050*, 2023.

[18] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, et al. Language modeling is compression. *arXiv preprint arXiv:2309.10668*, 2023.

[19] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[20] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[21] Byron Knoll. Cmix official website, 2024. [Online]. Available: https://github.com/byronknoll/cmix.

[22] NNCP. Nncp official website, 2024. [Online]. Available: https://bellard.org/nncp/.

[23] Byron Knoll. Data compression using lstm, 2017. [Online]. Available: https://github.com/byronknoll/lstm-compress.

[24] Valerii Likhosherstov, Krzysztof M Choromanski, Jared Quincy Davis, Xingyou Song, and Adrian Weller. Sub-linear memory: How to make performers slim. *Advances in Neural Information Processing Systems*, 34:6707–6719, 2021.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[26] Yu Mao, Yufei Cui, Tei-Wei Kuo, and Chun Jason Xue. Accelerating general-purpose lossless compression via simple and scalable parameterization. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 3205–3213, 2022.

[27] Ian H Witten, Radford M Neal, and John G Cleary. Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520–540, 1987.

[28] ZHONG Cheng and SUN Hui. Parallel algorithm for sensitive sequence recognition from long-read genome data with high error rate. *Journal on Communication/Tongxin Xuebao*, 44(2), 2023.

[29] Nicholas Wilt. *The cuda handbook: A comprehensive guide to gpu programming*. Pearson Education, 2013.

[30] Peter Pacheco. *An introduction to parallel programming*. Elsevier, 2011.

[31] Hui Sun, Yingfeng Zheng, Haonan Xie, Huidong Ma, Xiaoguang Liu, and Gang Wang. Pmffrc: a large-scale genomic short reads compression optimizer via memory modeling and redundant clustering. *BMC bioinformatics*, 24(1):454, 2023.

[32] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.

[33] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[34] Hanxiao Liu, Zihang Dai, David So, and Quoc V Le. Pay attention to mlps. *Advances in neural information processing systems*, 34:9204–9215, 2021.

[35] Sebastian Deorowicz. Silesia corpus offical. https://sun.aei.polsl.pl// sdeor/index.php?page=silesia, 1985.

[36] Diogo Pratas and Armando J Pinho. A dna sequence corpus for compression benchmark. In *Practical Applications of Computational Biology and Bioinformatics, 12th International Conference*, pages 208–215. Springer, 2019.

[37] Hui Sun, Yingfeng Zheng, Haonan Xie, Huidong Ma, Cheng Zhong, Meng Yan, Xiaoguang Liu, and Gang Wang. PQSDC: a parallel lossless compressor for quality scores data via sequences partition and Run-Length prediction mapping. *Bioinformatics*, page btae323, 05 2024.

[38] Hui Sun, Huidong Ma, Yingfeng Zheng, Haonan Xie, Meng Yan, Cheng Zhong, Xiaoguang Liu, and Gang Wang. Lrcb: A comprehensive benchmark evaluation of reference-free lossless compression tools for genomics sequencing long reads data. In *2024 Data Compression Conference (DCC)*, pages 584–584. IEEE, 2024.

[39] Hui Sun, Huidong Ma, Yingfeng Zheng, Haonan Xie, Xiaofei Wang, Xiaoguang Liu, and Gang Wang. Sr2c: A structurally redundant short reads collapser for optimizing dna data compression. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 60–67, 2023.

[40] Ravneet Kaur, Inderveer Chana, and Jhilik Bhattacharya. Data deduplication techniques for efficient cloud storage management: a systematic review. *The Journal of Supercomputing*, 74:2035–2085, 2018.

[41] Franck Cappello, Sheng Di, Sihuan Li, Xin Liang, Ali Murat Gok, Dingwen Tao, Chun Hong Yoon, Xin-Chuan Wu, Yuri Alexeev, and Frederic T Chong. Use cases of lossy compression for floating-point data in scientific data sets. *The International Journal of High Performance Computing Applications*, 33(6):1201–1220, 2019.

[42] Jürgen Schmidhuber and Stefan Heil. Sequential neural text compression. *IEEE Transactions on Neural Networks*, 7(1):142–146, 1996.

[43] Gzip. Gzip official website, 2018. [Online]. Available: http://www.gzip.org/.

[44] PBzip2. pbzip2 - parallel bzip2 file compressor, 2024. [Online]. Available: https://linux.die.net/man/1/pbzip2.

[45] XZ. Xz official website, 2024. [Online]. Available: https://github.com/tukaani-project/xz.

[46] Kubo Takehiro. Snzip, a compression/decompression tool based on snappy., 2021. [Online]. Available: https://github.com/kubo/snzip.

[47] LZMA. Lzma official website, 2008. [Online]. Available: https://tukaani.org/lzma/.

[48] Alistair Moffat. Implementing the PPM data compression scheme. *IEEE Trans. Commun.*, 38(11):1917–1921, 1990.

[49] LZ4. Lz4 official website. [Online]. Available: https://github.com/lz4/lz4.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 07:45:56 UTC from IEEE Xplore. Restrictions apply.